LEOPARDO RH

Dossier de Conception Technique

DCT v1.0 | Mars 2026 | Confidentiel

Document de référence pour Claude Code et Jules

Application	Leopardo RH — Plateforme SaaS RH Multi-Entreprises
Version DCT	1.0 — Initiale
Date	Mars 2026
Rédigé pour	Claude Code (backend) · Jules (frontend/mobile)
Stack	Laravel 11 · PostgreSQL 16 · Vue.js 3 · Flutter 3.x · Redis
Hébergement	o2switch (VPS Nginx + PHP-FPM)
Statut	VALIDÉ — Prêt pour exécution

1. OBJECTIF ET PORTÉE DU DCT

Le Dossier de Conception Technique (DCT) est le pont entre le Cahier des Charges fonctionnel (CDC v3.0) et le code source. Il répond à la question : comment, techniquement, allons-nous construire Leopardo RH ?

Ce document est la référence unique pour toute l'équipe de développement. Il doit être lu intégralement avant d'écrire la première ligne de code.

Ce document contient	Ce document ne contient PAS
Architecture système et décisions techniques	Le code source
Diagrammes d'architecture (composants, flux, données)	Les spécifications fonctionnelles (voir CDC v3.0)
Modèle de données complet (ERD)	Les maquettes UI (voir dossier wireframes)
Conventions de code et structure des projets	La documentation API Swagger (générée automatiquement)
Stratégie de déploiement sur o2switch	Les tests unitaires (rédigés pendant le développement)
Matrice des décisions techniques (ADR)	

2. ARCHITECTURE GLOBALE DU SYSTÈME

2.1 Vue d'Ensemble

Leopardo RH est une application SaaS multi-tenant organisée en quatre couches distinctes : les clients (navigateur web, app Flutter, lecteur ZKTeco), la passerelle API (Nginx), le backend applicatif (Laravel 11), et les services de données (PostgreSQL, Redis, stockage fichiers).

Couche	Composants	Responsabilité
Clients	Navigateur Web (Vue.js) · App Flutter (Android/iOS) · Lecteur ZKTeco	Interfaces utilisateur — consomment l'API REST
Passerelle	Nginx + SSL Let's Encrypt	Reverse proxy, load balancing, terminaison TLS, rate limiting
Backend	Laravel 11 (PHP 8.3) + Laravel Queues	Logique métier, API REST, génération PDF, envoi emails, jobs async
Cache & Queue	Redis 7	Cache des requêtes, sessions, file d'attente des jobs (paie, PDF, notifs)
Données	PostgreSQL 16 (schémas multi-tenant)	Persistance des données — 1 schéma isolé par entreprise
Stockage	Filesystem o2switch (S3-compatible possible)	Bulletins PDF, pièces jointes, photos de pointage (option)
Notifications	Firebase Cloud Messaging (FCM)	Notifications push vers l'app Flutter

2.2 Domaines et Sous-Domaines

Domaine	Usage	Pointe vers
leopardo-rh.com	Site vitrine / landing page	Page marketing (hors scope initial)
app.leopardo-rh.com	Interface web gestionnaire + employé	Nginx → Laravel + Vue.js
api.leopardo-rh.com	API REST consommée par Flutter et le web	Nginx → Laravel
admin.leopardo-rh.com	Panneau Super Admin	Nginx → Laravel (routes /admin)

2.3 Décisions d'Architecture (ADR)

> *Les ADR (Architecture Decision Records) documentent les choix techniques structurants avec leur contexte et leur justification. Chaque décision est irrévocable dans sa version — toute modification ouvre un nouveau ADR.*

ADR#	Décision	Justification	Alternatives rejetées
ADR-001	Multi-tenant par schéma PostgreSQL	Isolation totale des données, sauvegarde indépendante par entreprise, performances sans WHERE company_id	Shared schema (rejeté : risque de fuite), DB séparée (rejeté : complexité opérationnelle)
ADR-002	Laravel Sanctum pour l'auth (pas Passport)	Plus simple pour SPA + Mobile, tokens révocables par appareil, pas besoin d'OAuth complet	JWT stateless (rejeté : révocation complexe), Laravel Passport (rejeté : surcharge pour ce besoin)

ADR#	Décision	Justification	Alternatives rejetées
ADR-003	Flutter pour le mobile (pas React Native)	Performances natives, code unique Android/iOS, excellent support RTL arabe, écosystème stable	React Native (rejeté : bridge JS natif, perf variable), PWA seule (rejeté : limitations accès matériel)
ADR-004	Redis pour cache ET queues	Évite une dépendance supplémentaire, Redis gère les deux excellemment, disponible sur o2switch	RabbitMQ (rejeté : overkill), Database queues (rejeté : performances insuffisantes à l'échelle)
ADR-005	DomPDF pour les bulletins (pas wkhtmltopdf)	Pas de dépendance système externe, fonctionne sur o2switch mutualisé, suffisant pour bulletins standards	wkhtmltopdf (rejeté : nécessite accès système), Puppeteer (rejeté : Node.js requis en plus)
ADR-006	Inertia.js pour le web (pas API séparée)	Frontend Vue.js servi par Laravel sans duplication de routes, auth partagée, développement plus rapide	SPA découpléee (rejeté : double auth, CORS complexe), Blade seul (rejeté : UX insuffisante)
ADR-007	i18n dès le départ (pas ajouté après)	Retrofitter l'i18n coûte 3× le temps initial, arabe RTL nécessite des décisions de layout dès le début	Ajouter les langues en Phase 3 (rejeté : dette technique massive)

3. ARCHITECTURE MULTI-TENANT

3.1 Stratégie d'Isolation par Schéma PostgreSQL

Chaque entreprise cliente reçoit son propre schéma PostgreSQL au moment de la création de son compte. Le schéma public contient uniquement les tables partagées à l'échelle de la plateforme.

Schéma	Tables	Isolation
public	companies, plans, super_admins, invoices, billing_transactions, languages, hr_model_templates	Partagé — accessible uniquement par le Super Admin
company_{uuid}	employees, attendance_logs, absences, tasks, payrolls, settings... (toutes les tables RH)	Privé — accessible uniquement par les utilisateurs de cette entreprise

3.2 Résolution du Schéma Tenant

À chaque requête HTTP authentifiée, Laravel identifie le tenant actif et bascule la connexion PostgreSQL vers son schéma. Ce mécanisme est transparent pour toute la logique métier.

Étape	Mécanique	Code Laravel
1. Requête entrante	Le token Bearer est extrait du header Authorization	Sanctum middleware
2. Identification tenant	Le token est résolu → employee.company_id → companies.schema_name	TenantMiddleware::handle()
3. Connexion schéma	SET search_path TO company_{uuid} exécuté sur la connexion active	DB::statement('SET search_path TO ...')
4. Traitement	Tous les Eloquent models utilisent la connexion active — pas de WHERE company_id	Models standards
5. Isolation garantie	Impossible d'accéder aux données d'un autre tenant via l'API normale	Par construction PostgreSQL

3.3 Création d'un Nouveau Tenant

Quand le Super Admin crée une nouvelle entreprise, le système exécute automatiquement la séquence suivante :

1. Générer un UUID v4 unique pour l'entreprise
2. Insérer l'enregistrement dans public.companies avec schema_name = 'company_' + uuid
3. Exécuter CREATE SCHEMA company_{uuid} sur PostgreSQL
4. Exécuter les migrations Tenant sur ce nouveau schéma (php artisan tenants:migrate)
5. Appliquer le modèle RH du pays sélectionné (pré-remplir company_settings)
6. Créer le premier compte gestionnaire (hacher le MDP, stocker dans employees du schéma)
7. Envoyer l'email de bienvenue avec les identifiants

> *Outil recommandé : le package stancl/tenancy for Laravel gère la création de schéma, les migrations tenant et le switching automatique. Évaluer sa compatibilité avec l'approche PostgreSQL multi-schéma avant de démarrer.*

4. MODÈLE DE DONNÉES — ENTITÉS ET RELATIONS

4.1 Schéma Public — Tables Partagées

Table	Champs clés	Relations
plans	id, name, price_monthly, price_yearly, max_employees, features JSON, trial_days, is_active	1 plan → N companies
companies	id UUID PK, name, slug, sector, country, language, timezone, currency, plan_id FK, schema_name, status [active/suspended/trial/expired], subscription_start, subscription_end	N companies → 1 plan
super_admins	id, name, email, password_hash, last_login_at, 2fa_secret	Indépendant
invoices	id, company_id FK, amount, currency, period, status [draft/sent/paid/overdue], pdf_path, due_date, paid_at	N invoices → 1 company
billing_transactions	id, invoice_id FK, amount, gateway, gateway_ref, status, created_at	N transactions → 1 invoice
languages	id, code (fr/ar/tr/en...), name, direction (ltr/rtl), is_active	Référentiel global
hr_model_templates	id, country_code, name, cotisations JSON, ir_brackets JSON, leave_rules JSON, holiday_calendar JSON	Référentiel pays

4.2 Schéma company_{uuid} — Tables Tenant

4.2.1 Ressources Humaines Fondamentales

Table	Champs clés	Relations & Contraintes
employees	id, matricule (unique), first_name, last_name, email (unique), phone, password_hash, role [manager/employee], manager_role [principal/rh/dept/comptable/superviseur], department_id FK, position_id FK, schedule_id FK, manager_id FK self, site_id FK, contract_type, contract_start, contract_end, salary_base, salary_type, payment_method, iban, bank_account, leave_balance DECIMAL, status [active/suspended/archived], photo_path, emergency_contact JSON, extra_data JSON	Autoréférentielle sur manager_id. Clés étrangères vers departments, positions, schedules, sites
departments	id, name, manager_id FK (→ employees)	1 dept → N employees · 1 dept → 1 manager
positions	id, name, department_id FK	N positions → 1 department
schedules	id, name, start_time, end_time, break_minutes, work_days JSON ([1,2,3,4,5]), late_tolerance_minutes, overtime_threshold_daily, overtime_threshold_weekly, is_default BOOL	1 schedule → N employees

Table	Champs clés	Relations & Contraintes
sites	id, name, address, gps_lat, gps_lng, gps_radius_m	1 site → N employees

4.2.2 Pointage

Table	Champs clés	Notes
attendance_logs	id, employee_id FK, date DATE (unique avec employee_id), check_in TIMESTAMP, check_out TIMESTAMP NULL, declared_check_in TIME NULL, declared_check_out TIME NULL, method [mobile/qrcode/biometric/manual], gps_lat, gps_lng, gps_valid BOOL, photo_path NULL, hours_worked DECIMAL NULL, overtime_hours DECIMAL NULL, status [ontime/late/absent/leave/holiday/incomplete], is_manual_edit BOOL, edited_by FK NULL, edit_reason TEXT NULL	Index composé sur (employee_id, date). Contrainte UNIQUE sur (employee_id, date). L'horodatage serveur est toujours utilisé, jamais l'horloge client
devices	id, name, model, serial_number, type [zkteco/qrcode_terminal/tablet], site_id FK NULL, token (unique, haché), last_sync_at, status [active/inactive/error]	Token utilisé pour authentifier les pointages ZKTeco entrants

4.2.3 Absences et Congés

Table	Champs clés	Notes
absence_types	id, label, is_paid BOOL, pay_rate DECIMAL NULL, deducts_leave BOOL, requires_justification BOOL, min_notice_days INT, max_days INT NULL, who_submits [employee/manager], color VARCHAR(7)	Configurable librement par l'entreprise. Seeder avec types standards
absences	id, employee_id FK, type_id FK, start_date DATE, end_date DATE, days_count INT, status [pending/approved/rejected/cancelled], comment TEXT NULL, attachment_path NULL, decided_by FK NULL, decision_comment NULL, created_at	Calcul de days_count automatique (hors WE et jours fériés). Soft delete pour archivage
leave_balance_logs	id, employee_id FK, type [accrual/consumption/adjustment/reset], days DECIMAL, balance_after DECIMAL, reference_id NULL, created_by FK NULL, created_at	Audit complet de tous les mouvements de solde de congés

4.2.4 Avances sur Salaire

Table	Champs clés	Notes
salary_advances	id, employee_id FK, amount DECIMAL, reason TEXT NULL, status [pending/approved/rejected], repayment_plan JSON ([{month:'2026-04', amount:500}, ...]), amount_remaining DECIMAL, approved_by FK NULL, decision_comment NULL, created_at	repayment_plan est un tableau JSON. amount_remaining mis à jour automatiquement à chaque déduction sur paie

4.2.5 Tâches et Projets

Table	Champs clés	Notes
projects	id, name, description, start_date, end_date, members JSON, status [active/completed/archived]	Optionnel — regroupe des tâches
tasks	id, title, description, created_by FK, assigned_to JSON ([employee_ids]), project_id FK NULL, due_date DATETIME, priority [low/normal/high/urgent], status [todo/inprogress/review/done/rejected/cancelled], category VARCHAR NULL, checklist JSON ([{label, done}]), visibility [private/visible], created_at, updated_at	assigned_to est un tableau JSON d'IDs. Index sur (status, due_date) pour les alertes
task_comments	id, task_id FK, author_id FK, content TEXT, attachment_path NULL, created_at	Fil de discussion par tâche

4.2.6 Évaluations

Table	Champs clés	Notes
evaluations	id, employee_id FK, evaluator_id FK, period VARCHAR (ex : '2026-S1'), criteria_scores JSON ({criterion: score}), global_score DECIMAL, comment TEXT, objectives TEXT NULL, self_evaluation JSON NULL, created_at	self_evaluation rempli uniquement si l'option auto-évaluation est activée

4.2.7 Paie

Table	Champs clés	Notes
payrolls	id, employee_id FK, period_month INT, period_year INT, gross_salary DECIMAL, overtime_amount DECIMAL, bonuses JSON, deductions JSON, cotisations JSON, ir_amount DECIMAL, advance_deduction DECIMAL, absence_deduction DECIMAL, net_salary DECIMAL, pdf_path NULL, status [draft/validated], validated_by FK NULL, validated_at NULL	Contrainte UNIQUE sur (employee_id, period_month, period_year). Toutes les déductions sont stockées en JSON pour audit complet sur le bulletin
payroll_export_batches	id, period_month, period_year, bank_format VARCHAR, file_path, total_amount DECIMAL, employees_count INT, exported_by FK, created_at	Trace de chaque export bancaire généré

4.2.8 Paramètres et Audit

Table	Champs clés	Notes
company_settings	key VARCHAR PK, value TEXT, updated_at	Stockage clé-valeur pour tous les paramètres configurables. Clés préfixées par module (payroll.*, attendance.*, advance.*, etc.)
audit_logs	id, user_id FK, action VARCHAR, table_name VARCHAR, record_id, old_values JSON NULL, new_values JSON NULL, ip VARCHAR, user_agent TEXT, created_at	Rétention 24 mois. Index sur (user_id, created_at) et (table_name, record_id)
notifications	id, recipient_id FK, type VARCHAR, title VARCHAR, body TEXT, data JSON NULL, read_at TIMESTAMP NULL, created_at	Index sur (recipient_id, read_at) pour le compteur de non-lus

5. DIAGRAMMES DE FLUX — PROCESSUS CRITIQUES

> *Les diagrammes SVG interactifs sont disponibles dans le fichier compagnon HTML. Cette section décrit les flux en texte structuré pour référence dans le document Word.*

5.1 Flux d'Authentification Multi-Tenant

Ce flux décrit le chemin complet d'une requête authentifiée depuis le client jusqu'à la base de données tenant.

Étape	Acteur	Action	Résultat
1	Client (Flutter/Web)	Envoie POST /auth/login avec email + password	—
2	Nginx	Passe la requête à Laravel (rate limit : 5 tentatives/min/IP)	—
3	Laravel AuthController	Cherche l'email dans public.companies JOIN employees de tous les schémas → NON — cherche dans public.companies, récupère schema_name	Token Sanctum créé si succès
4	Laravel AuthController	SET search_path TO company_{uuid}, vérifie le mot de passe, crée le token Sanctum	Token renvoyé au client
5	Client	Stocke le token (Flutter : flutter_secure_storage / Web : httpOnly cookie)	—
6	Client	Toute requête suivante : Authorization: Bearer {token}	—
7	TenantMiddleware	Résout le token → employee → company → schema_name, SET search_path	Connexion tenant active
8	Controller	Exécute la logique métier — models Eloquent dans le bon schéma automatiquement	Réponse JSON

5.2 Flux de Pointage Déclaratif (Mobile Flutter)

Étape	Acteur	Action	Condition / Résultat
1	Employé	Ouvre l'app Flutter — session active (token en mémoire sécurisée)	Si token expiré → écran de reconnexion
2	App Flutter	Affiche le bouton 'Pointer mon arrivée' ou 'Pointer mon départ' selon le dernier pointage	Logique : pas de check_in aujourd'hui → bouton Arrivée
3	Employé	Appuie sur le bouton et confirme	—
4	App Flutter	Si GPS activé → demande la position, si photo activée → ouvre caméra	Erreur GPS : refus avec message
5	App Flutter	POST /attendance/check-in ou /check-out avec {timestamp_local, gps?, photo_base64?}	Timeout 10s → message d'erreur offline
6	AttendanceController	Utilise TOUJOURS NOW() côté serveur, ignore le timestamp client	Protection contre manipulation de l'horloge
7	AttendanceController	Vérifie : GPS dans la zone ? (si activé), photo valide ? Enregistre l'attendance_log	GPS hors zone → rejeté avec message

Étape	Acteur	Action	Condition / Résultat
8	AttendanceController	Calcule le statut (ontime/late) selon le planning de l'employé	Retard → notification push au gestionnaire
9	App Flutter	Affiche la confirmation avec l'heure enregistrée (heure serveur)	Notification push de confirmation à l'employé

5.3 Flux du Calcul de Paie Mensuel

Étape	Acteur	Action	Notes techniques
1	Gestionnaire	Accède au module Paie → sélectionne le mois	—
2	PayrollController	Agrège pour chaque employé : jours travaillés, heures sup, absences non payées, avances dues, primes	Job async via Redis queue si > 50 employés
3	PayrollService	Applique la formule : BRUT → cotisations → IR → retenues → NET	Les paramètres (taux, tranches) lus depuis company_settings
4	Gestionnaire	Visualise la simulation (avant validation) — peut corriger des anomalies	Statut payroll = 'draft'
5	Gestionnaire	Clique 'Valider la paie' — décision irréversible	Confirmation double-click obligatoire
6	PayrollService	Génère tous les bulletins PDF (DomPDF) en queue	1 job par employé, parallélisé via Redis
7	PayrollService	Met à jour salary_advances (amount_remaining) pour les remboursements du mois	Transaction atomique PostgreSQL
8	PayrollService	Stocke les PDF, met à jour payrolls.status = 'validated'	—
9	NotificationService	Envoie une notification push + email à chaque employé : 'Votre bulletin est disponible'	—
10	Gestionnaire	Génère le fichier d'export bancaire (CSV/XML selon format configuré)	Trace dans payroll_export_batches

5.4 Flux de Demande de Congé

Étape	Acteur	Action	Règles
1	Employé (Flutter)	Sélectionne le type de congé, dates de début et fin	Calcul auto des jours ouvrables hors WE et fériés
2	App Flutter	Vérifie : solde suffisant ? Délai de prévenance respecté ? Pièce jointe requise ?	Erreurs affichées avant envoi
3	App Flutter	POST /absences avec les données validées	—
4	AbsenceController	Vérifie à nouveau les règles côté serveur (solde, délai)	Double validation client + serveur
5	AbsenceController	Crée l'absence en statut 'pending', déduit provisoirement le solde	—
6	NotificationService	Notifie le gestionnaire : push + web + email selon matrice	—

Étape	Acteur	Action	Règles
7	Gestionnaire	Consulte la demande avec : solde actuel, planning équipe sur la période, historique demandes	—
8a	Gestionnaire (approbation)	PUT /absences/{id}/approve → solde défalqué définitivement, statut = 'approved'	Notification push + email à l'employé
8b	Gestionnaire (refus)	PUT /absences/{id}/reject avec motif obligatoire → solde restauré, statut = 'rejected'	Notification push + email à l'employé avec motif
9	AttendanceService	Marque automatiquement les jours concernés dans attendance_logs comme statut 'leave'	Synchronisation automatique paie ↔ pointage

6. STRUCTURE DES PROJETS ET CONVENTIONS DE CODE

6.1 Repository Backend — leopardo-rh-api

```
leopardo-rh-api/ ├── app/ │   ├── Http/ │   │   ├── Controllers/ │   │   │   ├── Auth/
→ AuthController.php │   │   │   ├── Admin/          → CompanyController,
PlanController, BillingController │   │   │   ├── Tenant/         → EmployeeController,
AttendanceController, AbsenceController, │   │   │   │
PayrollController, TaskController, AdvanceController, ... │   │   │   └── Shared/
→ NotificationController, ProfileController │   │   ├── Middleware/ │   │   │   ├──
TenantMiddleware.php        → SET search_path TO company_{uuid} │   │   │   ├──
SuperAdminMiddleware.php    → Vérifie le rôle super_admin │   │   │   └──
ManagerMiddleware.php       → Vérifie le rôle manager │   │   └── Requests/           →
FormRequest par action (validation centralisée) │   ├── Models/ │   │   ├── Public/
→ Company.php, Plan.php, SuperAdmin.php, Invoice.php │   │   └── Tenant/             →
Employee.php, AttendanceLog.php, Absence.php, ... │   ├── Services/               →
PayrollService, NotificationService, BankExportService, ... │   ├── Jobs/
→ GeneratePayslipPDF, SendBulkNotification, SyncZKTeco │   └── Observers/              →
EmployeeObserver (audit_log auto), AbsenceObserver ├── database/ │   ├── migrations/ │
│   ├── public/             → Migrations du schéma partagé │   │   └── tenant/
→ Migrations du schéma tenant (exécutées par tenancy) │   └── seeders/ │       ├──
HRModelSeeder.php   → Pré-charge les modèles RH par pays │       └── LanguageSeeder.php
→ Pré-charge FR, AR, TR, EN ├── routes/ │   ├── api.php                → Routes API
publiques (auth) │   ├── admin.php              → Routes Super Admin │   └── tenant.php
→ Routes tenant (gestionnaire + employé) ├── resources/lang/            → fr/, ar/,
tr/, en/ (fichiers de traduction) └── config/tenancy.php          → Configuration du
multi-tenant
```

6.2 Repository Mobile — leopardo-rh-mobile (Flutter)

```
leopardo-rh-mobile/ ├── lib/ │   ├── main.dart              → Entry point, MaterialApp
+ L10n setup │   ├── core/ │   │   ├── api/               → ApiClient (Dio),
interceptors (auth, refresh token) │   │   ├── auth/               → AuthBloc ou
Riverpod auth provider │   │   ├── i18n/              → AppLocalizations, RTL detection
│   │   └── storage/            → FlutterSecureStorage wrapper │   ├── features/ │   │
├── auth/               → LoginScreen, ForgotPasswordScreen │   │   ├── dashboard/
→ EmployeeDashboard, ManagerDashboard │   │   ├── attendance/         → AttendanceScreen
(grand bouton), QRScanner, History │   │   ├── absences/           → AbsenceListScreen,
AbsenceRequestScreen │   │   ├── tasks/              → TaskListScreen, TaskDetailScreen,
CommentThread │   │   ├── advances/           → AdvanceScreen, RepaymentPlanScreen │   │
├── payroll/            → PayslipListScreen, PayslipViewerScreen │   │   ├── employees/
→ EmployeeListScreen, EmployeeFormScreen (manager) │   │   └── profile/            →
ProfileScreen, NotificationPreferencesScreen │   └── shared/ │       ├── widgets/
→ Composants réutilisables (LeopardoButton, StatusBadge...) │       └── theme/
→ LeopardoTheme (couleurs, typo, dark mode) ├── assets/ │   ├── l10n/
→ app_fr.arb, app_ar.arb, app_tr.arb, app_en.arb │   └── images/                → Logo,
icônes └── pubspec.yaml                → Dépendances (Dio, Riverpod/BLoC,
flutter_localizations,                                          qr_code_scanner,
flutter_secure_storage, FCM...)
```

6.3 Repository Web — leopardo-rh-web (Vue.js + Inertia)

```
leopardo-rh-web/ (intégré dans leopardo-rh-api/resources/) ├── js/ │   ├── Pages/ │   │
├── Auth/               → Login.vue, ForgotPassword.vue │   │   ├── Admin/
→ Companies/Index.vue, Companies/Create.vue, Plans/... │   │   ├── Dashboard/          →
ManagerDashboard.vue, AttendanceLive.vue │   │   ├── Employees/          → Index.vue,
Create.vue, Show.vue │   │   ├── Attendance/         → Calendar.vue, History.vue │   │
├── Absences/           → Index.vue, Calendar.vue │   │   ├── Payroll/            →
Index.vue, Calculate.vue, Validate.vue │   │   └── Settings/           → Company.vue,
HR.vue, Payroll.vue, Devices.vue │   ├── Components/             → Composants
réutilisables Vue.js │   └── Layouts/ │       ├── AppLayout.vue       → Layout principal
gestionnaire (sidebar + header) │       └── AdminLayout.vue     → Layout Super Admin
```

7. CONVENTIONS DE CODE ET STANDARDS

7.1 Conventions Laravel (Backend)

Élément	Convention	Exemple
Nommage Models	PascalCase, singulier	Employee, AttendanceLog, AbsenceType
Nommage Tables	snake_case, pluriel	employees, attendance_logs, absence_types
Nommage Controllers	PascalCase + Controller	EmployeeController, PayrollController
Nommage Services	PascalCase + Service	PayrollService, BankExportService
Nommage Jobs	Action + Nom	GeneratePayslipPDF, SendPayrollNotification
Nommage Routes	kebab-case	/attendance/check-in, /payroll/validate
Réponse API success	{ data: {...}, message: 'Success' }	HTTP 200/201
Réponse API error	{ error: 'Code', message: 'Description', details: {} }	HTTP 4xx/5xx
Validation	FormRequest systématiquement — jamais dans le Controller	StoreEmployeeRequest
Eloquent	Toujours utiliser les relations Eloquent — jamais de raw SQL sauf cas exceptionnel	Employee::with('department')->get()
Transactions	DB::transaction() pour toute opération multi-table	Paie, validation congé, approbation avance
Dates/Heures	Toujours UTC en base, formater selon le fuseau de l'entreprise dans l'API	Carbon avec timezone company

7.2 Format de Réponse API Standard

```
// Succès (liste) {   "data": [ { "id": 1, "first_name": "Ahmed", ... }, ... ],
"meta": { "total": 45, "per_page": 15, "current_page": 1, "last_page": 3 } }  // Succès
(ressource unique) {   "data": { "id": 1, "first_name": "Ahmed", "last_name": "Benali",
... },   "message": "Employé créé avec succès" }  // Erreur de validation {   "error":
"VALIDATION_ERROR",   "message": "Les données fournies sont invalides",   "details": {
"email": ["L'email est déjà utilisé"], "salary_base": ["Doit être positif"] } }  //
Erreur métier {   "error": "INSUFFICIENT_LEAVE_BALANCE",   "message": "Solde de congés
insuffisant (disponible: 3 jours, demandés: 5 jours)" }
```

7.3 Conventions Flutter (Mobile)

Élément	Convention	Exemple
Gestion d'état	Riverpod (préféré) ou BLoC — choisir un seul pattern pour tout le projet	EmployeeNotifier extends StateNotifier
Nommage fichiers	snake_case	attendance_screen.dart, employee_card.dart
Nommage classes	PascalCase	AttendanceScreen, EmployeeCard

Élément	Convention	Exemple
Nommage variables	camelCase	currentEmployee, isLoading
API calls	Repository pattern — jamais d'appels Dio directs dans les widgets	EmployeeRepository.getAll()
Internationalisation	AppLocalizations.of(context).labelName — jamais de string hardcodée	context.l10n.checkIn
RTL support	Directionality widget à la racine — utiliser TextDirection.of(context)	Testé sur Arabe obligatoirement
Navigation	GoRouter — routes nommées, pas de Navigator.push direct	context.go('/attendance')
Erreurs réseau	Afficher un message clair à l'utilisateur — jamais de crash silencieux	SnackBar avec retry button

8. INTERNATIONALISATION (i18n) — IMPLÉMENTATION

8.1 Architecture i18n Backend (Laravel)

Toutes les chaînes utilisateur sont dans les fichiers de langue. Le backend retourne les clés de traduction dans ses réponses JSON — le client traduit selon sa locale active.

Fichier	Contenu	Exemple de clé
resources/lang/fr/messages.php	Toutes les chaînes en français	'check_in_success' => 'Pointage d\'arrivée enregistré'
resources/lang/ar/messages.php	Traduction arabe (RTL)	'check_in_success' => 'تم تسجيل الحضور'
resources/lang/tr/messages.php	Traduction turque	'check_in_success' => 'Giriş saati kaydedildi'
resources/lang/en/messages.php	Traduction anglaise	'check_in_success' => 'Check-in recorded successfully'

La langue active est déterminée par le champ language de la company. Middleware SetLocale lit ce champ et appelle App::setLocale() à chaque requête tenant.

8.2 Architecture i18n Mobile (Flutter)

```
// pubspec.yaml dependencies:   flutter_localizations:     sdk: flutter   intl: ^0.19.0
// l10n.yaml arb-dir: assets/l10n template-arb-file: app_fr.arb output-class:
AppLocalizations  // assets/l10n/app_fr.arb {   "@@locale": "fr",   "checkIn": "Pointer
mon arrivée",   "checkOut": "Pointer mon départ",   "leaveBalance": "Solde de congés :
{days} jour(s)",   "@leaveBalance": { "placeholders": { "days": { "type": "int" } } } }
// Utilisation dans un widget Text(AppLocalizations.of(context)!.checkIn) // ou avec
extension contextuelle Text(context.l10n.checkIn)  // RTL automatique selon la locale
MaterialApp(   locale: Locale(companyLanguage), // 'ar', 'fr', etc.   supportedLocales:
AppLocalizations.supportedLocales,   localizationsDelegates:
AppLocalizations.localizationsDelegates, )
```

8.3 Ajouter une Nouvelle Langue — Procédure

8. Créer resources/lang/{code}/messages.php (copier fr, traduire les valeurs)
9. Créer assets/l10n/app_{code}.arb (copier app_fr.arb, traduire les valeurs)
10. Ajouter la locale dans MaterialApp.supportedLocales côté Flutter
11. Insérer la langue dans la table public.languages (is_active = false)
12. Le Super Admin active la langue depuis son panneau — disponible pour tous les gestionnaires

> *Pas de redéploiement nécessaire pour activer une langue déjà traduite — activation via l'interface Super Admin.*

9. SÉCURITÉ — IMPLÉMENTATION DÉTAILLÉE

9.1 Authentification — Laravel Sanctum

Scénario	Mécanique	Durée token
Connexion web (Vue.js)	Cookie httpOnly SameSite=Strict — pas de token en JS	Session (8h d'inactivité)
Connexion mobile (Flutter)	Token opaque stocké dans flutter_secure_storage	90 jours, révocable
Super Admin	Token distinct, non lié à un tenant	4h avec 2FA obligatoire
Lecteur ZKTeco	Token device statique, non révocable en prod (changé manuellement)	Permanent jusqu'au renouvellement

9.2 Matrice des Permissions

Action	Super Admin	Manager Principal	Manager RH	Manager Dept	Employé
Créer une entreprise	✓	—	—	—	—
Voir toutes les entreprises	✓	—	—	—	—
CRUD employés	—	✓	✓	Son dept	—
Valider congés	—	✓	✓	Son dept	—
Corriger pointages	—	✓	✓	Son dept	—
Générer la paie	—	✓	✓	—	—
Exporter virement bancaire	—	✓	Comptable	—	—
Pointer (arrivée/départ)	—	—	—	—	✓
Voir ses propres données	—	—	—	—	✓
Soumettre demande congé/avance	—	—	—	—	✓
Télécharger son bulletin	—	—	—	—	✓
Voir données autres employés	—	—	—	—	✗ (jamais)

9.3 Protection Contre les Attaques Courantes

Attaque	Protection mise en place
Injection SQL	Éloquent ORM avec bindings préparés — jamais de raw SQL avec input utilisateur
Cross-Site Scripting (XSS)	Vue.js échappe automatiquement les données. Inertia ne rend pas de HTML brut.
Cross-Site Request Forgery (CSRF)	Token CSRF Laravel pour le web. Mobile utilise Bearer token (pas vulnérable au CSRF).
Brute Force	5 tentatives de connexion → blocage 15 min. Rate limit Nginx : 60 req/min par IP.
Mass Assignment	Tous les models Laravel ont $fillable explicite — jamais $guarded = []

Attaque	Protection mise en place
Tenant Isolation Breach	TenantMiddleware vérifié sur toutes les routes tenant — impossible de switcher de schéma
Token Leak	flutter_secure_storage chiffré (Keystore Android / Keychain iOS). Web : cookie httpOnly.
Données sensibles exposées	RIB/IBAN chiffrés avec Laravel Crypt. Photo de pointage : chemin non devinable (UUID).

10. INTÉGRATION BIOMÉTRIQUE ZKTECO

10.1 Modes d'Intégration

Mode	Description	Quand utiliser
Push (primaire)	Le lecteur ZKTeco envoie les pointages à notre webhook API en temps réel dès qu'un employé pose son doigt	Réseau local stable entre le lecteur et Internet
Pull (fallback)	Laravel interroge le lecteur toutes les 60 secondes via sa propre API pour récupérer les logs non encore envoyés	En cas de panne Push, ou pour la synchronisation initiale

10.2 Flux d'Intégration Push

13. ZKTeco configuré avec l'URL webhook : https://api.leopardo-rh.com/v1/attendance/biometric
14. À chaque pointage : le lecteur envoie POST avec {device_token, employee_id, timestamp, direction}
15. BiometricController vérifie le device_token haché contre la table devices
16. L'employee_id ZKTeco est mappé vers notre employees.id via le champ extra_data JSON
17. L'attendance_log est créé exactement comme un pointage mobile normal
18. Si le timestamp ZKTeco et le timestamp serveur diffèrent de plus de 5 minutes → alerte audit

10.3 Enrôlement des Employés sur ZKTeco

L'enrôlement biométrique (empreintes) se fait physiquement sur le lecteur. La plateforme synchronise uniquement les métadonnées (nom, ID) vers le lecteur via son API locale pour que les noms s'affichent correctement sur son écran LCD.

> *Les données biométriques (empreintes digitales) ne quittent jamais le lecteur ZKTeco. Leopardo RH ne stocke et ne traite aucune donnée biométrique — seulement l'identifiant et l'horodatage.*

11. DÉPLOIEMENT ET INFRASTRUCTURE — O2SWITCH

11.1 Configuration VPS o2switch Recommandée

Composant	Configuration	Notes
OS	Ubuntu 22.04 LTS	Support long terme, compatible tout l'écosystème
PHP	PHP 8.3 avec extensions : pdo_pgsql, redis, gd, zip, mbstring, curl, bcmath	FPM pour les performances
PostgreSQL	PostgreSQL 16	Disponible sur o2switch VPS
Nginx	Nginx 1.24+	Serveur web + reverse proxy
Redis	Redis 7 (socket Unix local)	Cache + queues
SSL	Let's Encrypt via Certbot (renouvellement auto)	Gratuit, renouvelé automatiquement
Process Manager	Supervisor pour les Laravel workers (queues)	Redémarrage automatique si crash

11.2 Configuration Nginx

```
# /etc/nginx/sites-available/leopardo-api server {     listen 443 ssl http2;
server_name api.leopardo-rh.com;     root /var/www/leopardo-rh-api/public;     index
index.php;      ssl_certificate /etc/letsencrypt/live/api.leopardo-rh.com/fullchain.pem;
ssl_certificate_key /etc/letsencrypt/live/api.leopardo-rh.com/privkey.pem;      # Rate
limiting : 60 req/min par IP     limit_req_zone $binary_remote_addr zone=api:10m
rate=1r/s;     limit_req zone=api burst=60 nodelay;      location / {         try_files
$uri $uri/ /index.php?$query_string;     }      location ~ \.php$ {         fastcgi_pass
unix:/var/run/php/php8.3-fpm.sock;         fastcgi_param SCRIPT_FILENAME
$realpath_root$fastcgi_script_name;         include fastcgi_params;     }      #
Sécurité : bloquer l'accès aux fichiers sensibles     location ~ /\. { deny all; }
location ~ /(storage|bootstrap/cache) { deny all; } }
```

11.3 Configuration Supervisor (Laravel Queues)

```
# /etc/supervisor/conf.d/leopardo-worker.conf [program:leopardo-worker]
process_name=%(program_name)s_%(process_num)02d command=php /var/www/leopardo-rh-
api/artisan queue:work redis     --sleep=3 --tries=3 --max-time=3600 autostart=true
autorestart=true stopasgroup=true killasgroup=true user=www-data numprocs=3   ; 3
workers pour paralléliser la génération PDF de paie redirect_stderr=true
stdout_logfile=/var/www/leopardo-rh-api/storage/logs/worker.log
```

11.4 Cron Jobs

Fréquence	Commande Artisan	Rôle
Toutes les minutes	schedule:run	Déclenche le scheduler Laravel (tous les jobs planifiés)
Toutes les heures	attendance:check-missing	Alerte les gestionnaires pour les pointages manquants
Chaque jour à 00h01	leave:accrue-monthly	Calcule l'acquisition mensuelle des congés (1er du mois)
Chaque jour à 00h30	backup:run --only-db	Sauvegarde PostgreSQL complète
Chaque jour à 07h00	contracts:expiry-alerts	Alertes contrats CDD et périodes d'essai

Fréquence	Commande Artisan	Rôle
Chaque lundi à 08h00	tasks:overdue-alerts	Alertes tâches en retard de la semaine
1er du mois à 06h00	subscription:expiry-alerts	Alertes abonnements expirant dans 30j, 15j, 7j
Toutes les minutes	zkteco:pull-sync	Récupération fallback des pointages ZKTeco non reçus en Push

11.5 Stratégie de Sauvegarde

Type	Fréquence	Rétention	Méthode
Dump PostgreSQL complet	Quotidienne (00h30)	30 jours	pg_dump → compressé → stocké /backup/daily/
Dump PostgreSQL incrémental	Toutes les heures	7 jours	pg_dump schémas modifiés uniquement
Fichiers applicatifs (storage)	Quotidienne	7 jours	rsync vers répertoire de sauvegarde o2switch
Sauvegarde distante	Hebdomadaire	3 mois	SFTP vers serveur distinct (idéalement hors o2switch)

12. MODULE EXPORT BANCAIRE — VIREMENT MASSE SALARIALE

12.1 Principe

Le BankExportService génère un fichier structuré contenant tous les virements salariaux du mois validé. Le format varie selon la banque configurée par l'entreprise. Ce fichier est remis au responsable comptable qui le soumet à la banque via son interface de banque en ligne professionnelle.

> *Le fichier généré est une simulation basée sur les formats standards documentés publiquement des banques locales. Il doit être validé avec la banque de l'entreprise avant le premier virement réel.*

12.2 Formats Supportés

Identifiant	Banque / Région	Format	Structure
DZ_GENERIC	Algérie (CNEP, BEA, BNA, CPA)	CSV délimité point-virgule	Code banque ; RIB (20 chiffres) ; Montant ; Libellé ; Date valeur
MA_CIH	Maroc — CIH Bank	CSV UTF-8	IBAN ; BIC ; Montant MAD ; Libellé ; Référence virement
MA_ATTIJARIWAFA	Maroc — Attijariwafa Bank	CSV ISO-8859-1	N° compte ; Montant ; Nom bénéficiaire ; Référence
TN_GENERIC	Tunisie (STB, BIAT, BNA)	TXT colonnes fixes	RIB TN (20 car.) + Montant (12 car. droite) + Libellé (35 car.)
FR_SEPA	France + Zone Euro	XML ISO 20022 pain.001.001.03	Standard SEPA — IBAN + BIC + Amount + Creditor
GENERIC_CSV	Configurable par Super Admin	CSV colonnes configurables	Mapping des colonnes définissable dans les paramètres

```
// Exemple CSV DZ_GENERIC // En-tête : Code banque ; RIB ; Montant ; Libellé ; Date
valeur 020;00200601234567890123;45000.00;SALAIRE AVRIL 2026 - BENALI AHMED;30/04/2026
020;00200601234567890456;38500.00;SALAIRE AVRIL 2026 - MEZIANI SARA;30/04/2026  //
Exemple XML FR_SEPA (pain.001) <?xml version="1.0" encoding="UTF-8"?> <Document
xmlns="urn:iso:std:iso:20022:tech:xsd:pain.001.001.03">   <CstmrCdtTrfInitn>
<GrpHdr>       <MsgId>LEOPARDO-2026-04-001</MsgId>       <CreDtTm>2026-04-
30T10:00:00</CreDtTm>       <NbOfTxs>25</NbOfTxs>       <CtrlSum>987650.00</CtrlSum>
</GrpHdr>     ...   </CstmrCdtTrfInitn> </Document>
```

13. PERFORMANCE ET SCALABILITÉ

13.1 Stratégie de Cache Redis

Donnée cachée	Clé Redis	TTL	Invalidation
Paramètres entreprise (company_settings)	tenant:{uuid}:settings	1h	À chaque modification de settings
Planning de travail actif	tenant:{uuid}:schedules	24h	À chaque modification de schedule
Liste des départements	tenant:{uuid}:departments	24h	À chaque CRUD département
Jours fériés du pays	holidays:{country}:{year}	365j	Jamais (données statiques)
Token → company mapping	auth:token:{hash}	Token TTL	À la déconnexion / révocation
Statistiques dashboard	tenant:{uuid}:dashboard:{date}	5 min	Automatique par TTL

13.2 Index de Base de Données Critiques

Table	Index	Justification
attendance_logs	UNIQUE (employee_id, date) + INDEX (date, status)	Requêtes de présence quotidienne et vue temps réel
employees	INDEX (department_id) + INDEX (status) + UNIQUE (email)	Filtres par département et recherche par email
absences	INDEX (employee_id, status) + INDEX (start_date, end_date)	Vérification solde et calendrier des absences
tasks	INDEX (assigned_to) + INDEX (status, due_date)	Mes tâches et alertes de retard
payrolls	UNIQUE (employee_id, period_month, period_year)	Contrainte d'unicité bulletin par période
audit_logs	INDEX (user_id, created_at) + INDEX (table_name, record_id)	Requêtes d'audit fréquentes
notifications	INDEX (recipient_id, read_at)	Compteur de non-lus en temps réel

13.3 Objectifs de Performance

Endpoint	Cible p95	Stratégie
POST /attendance/check-in	< 150ms	Pas de cache — écriture directe. Validation minimale.
GET /dashboard (gestionnaire)	< 300ms	Cache Redis 5 min. Agrégation pré-calculée.
POST /payroll/calculate (500 emp.)	< 30s async	Job Redis, résultat en polling ou WebSocket
GET /employees (paginé 15)	< 200ms	Index département. Eager loading limité.
GET /reports/* (export Excel)	< 60s async	Job Redis, téléchargement par lien signé temporaire

14. PLAN D'EXÉCUTION — SPRINT 0 (AVANT LE CODE)

14.1 Checklist Pré-développement

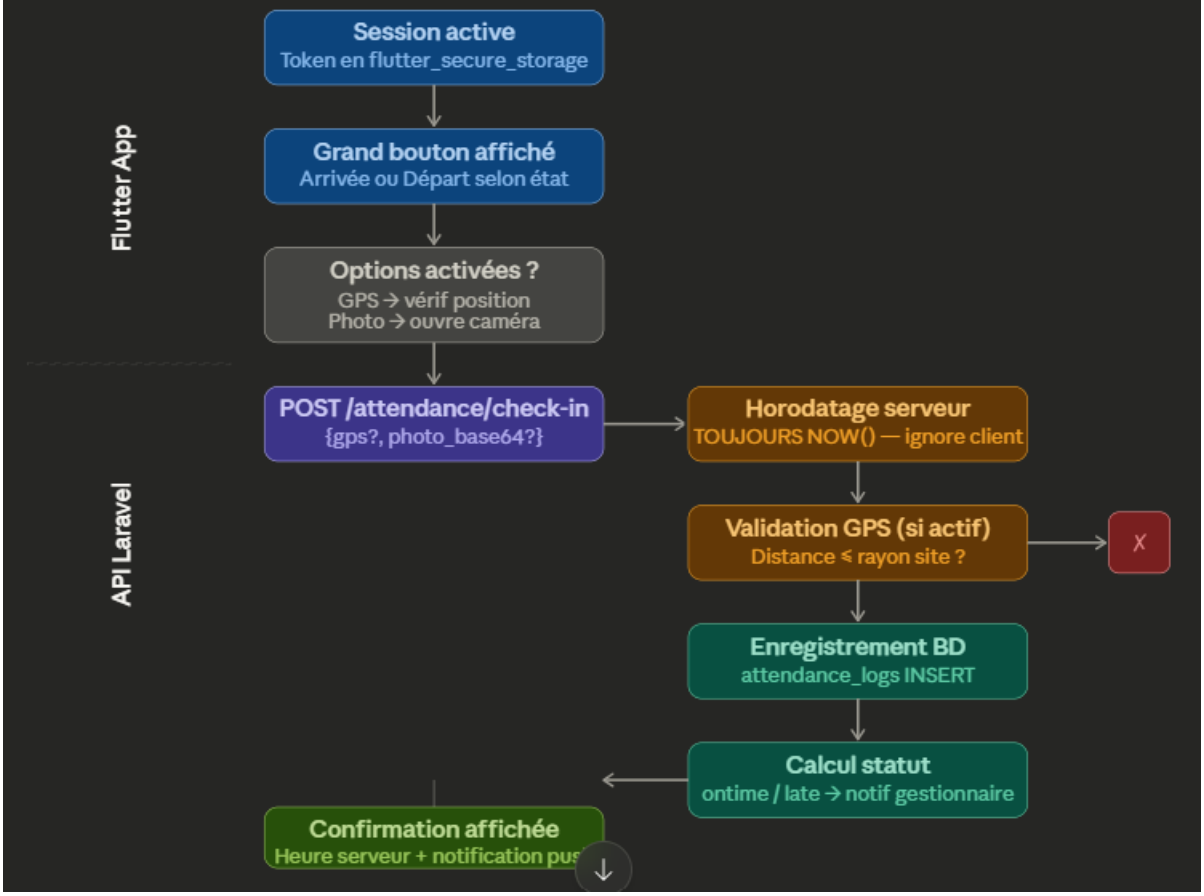
#	Tâche	Responsable	Statut
1	Achat domaine leopardo-rh.com + configuration DNS o2switch	Client	À faire
2	Création compte o2switch VPS + configuration SSH	Client	À faire
3	Création projet Firebase (FCM pour notifications push)	Équipe dev	À faire
4	Création compte Google Play Developer (25$ unique)	Client	À faire
5	Création compte Apple Developer (99$/an)	Client	À faire
6	Création des 3 repositories Git (api, mobile, web)	Équipe dev	À faire
7	Installation PHP 8.3 + PostgreSQL 16 + Redis + Nginx sur o2switch VPS	Équipe dev	À faire
8	Initialisation projet Laravel 11 avec stancl/tenancy	Claude Code	À faire
9	Création fichiers de traduction initiaux (fr, ar, tr, en) — même partiels	Équipe dev	À faire
10	Initialisation projet Flutter avec GoRouter + Riverpod + l10n	Jules	À faire
11	Configuration CI/CD basique (déploiement Git → o2switch)	Équipe dev	À faire
12	Validation du DCT par toute l'équipe avant démarrage codage	Tous	EN COURS

14.2 Ordre de Développement — Phase 1 MVP

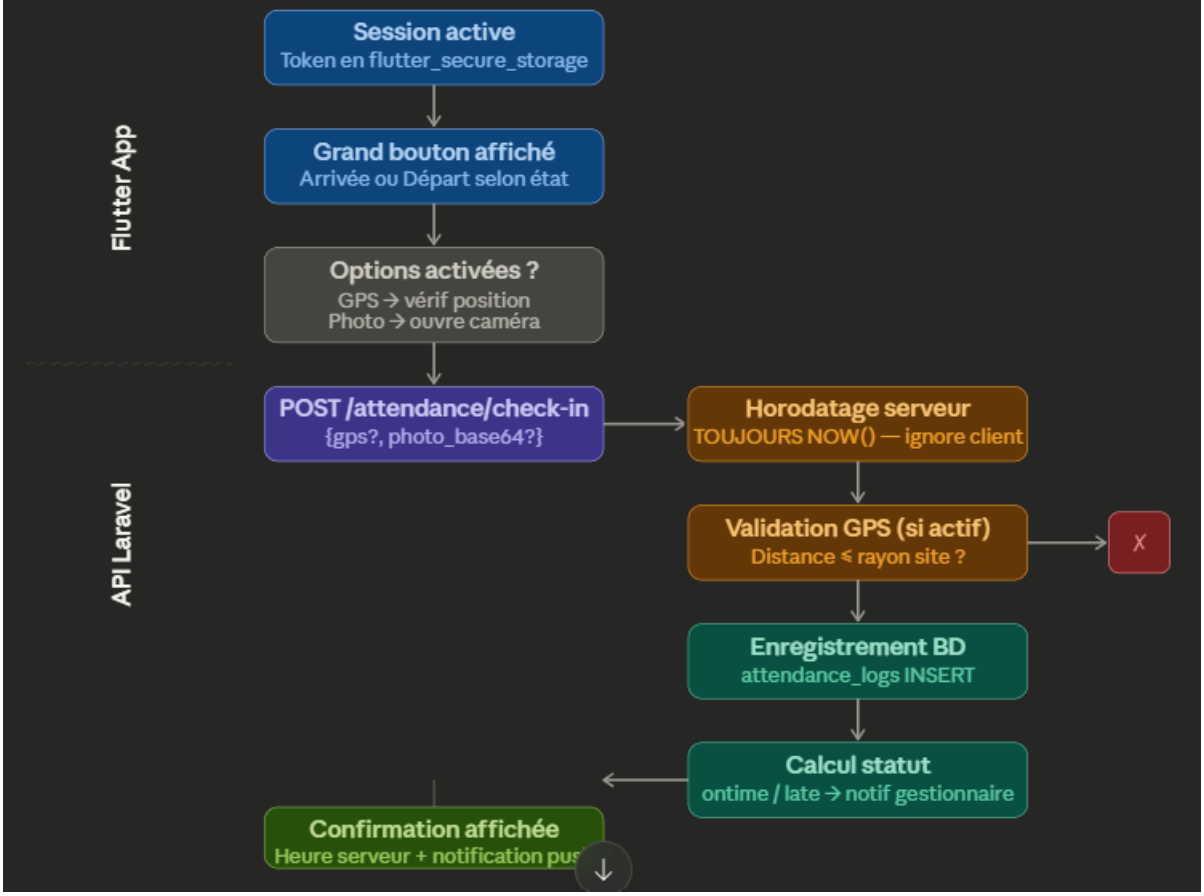
Semaine	Module Backend (Claude Code)	Module Mobile (Jules)	Module Web (Optionnel)
S1-S2	Infrastructure Laravel + PostgreSQL multi-tenant + migrations public	Initialisation Flutter + auth screens + navigation GoRouter	Layout de base + login
S3-S4	API Auth (Sanctum) + TenantMiddleware + i18n backend	Écran de pointage déclaratif + API integration + i18n Flutter	Dashboard vide
S5-S6	CRUD Employés + import CSV + alertes contrats	Écrans gestion employés + fiche employé (vue employé)	CRUD employés web
S7-S8	Module Absences (types + flux demande + validation)	Écran mes absences + formulaire demande + solde	Calendrier absences web
S9-S10	Calcul paie basique (fixe mensuel) + DomPDF bulletins	Écran mes bulletins + téléchargement PDF	Module paie web
S11-S12	Notifications (FCM push + email) + Super Admin + facturation manuelle	Notifications push + écran Super Admin (si applicable)	Tableau de bord gestionnaire complet

Leopardo RH — DCT v1.0 — Document de conception finalisé

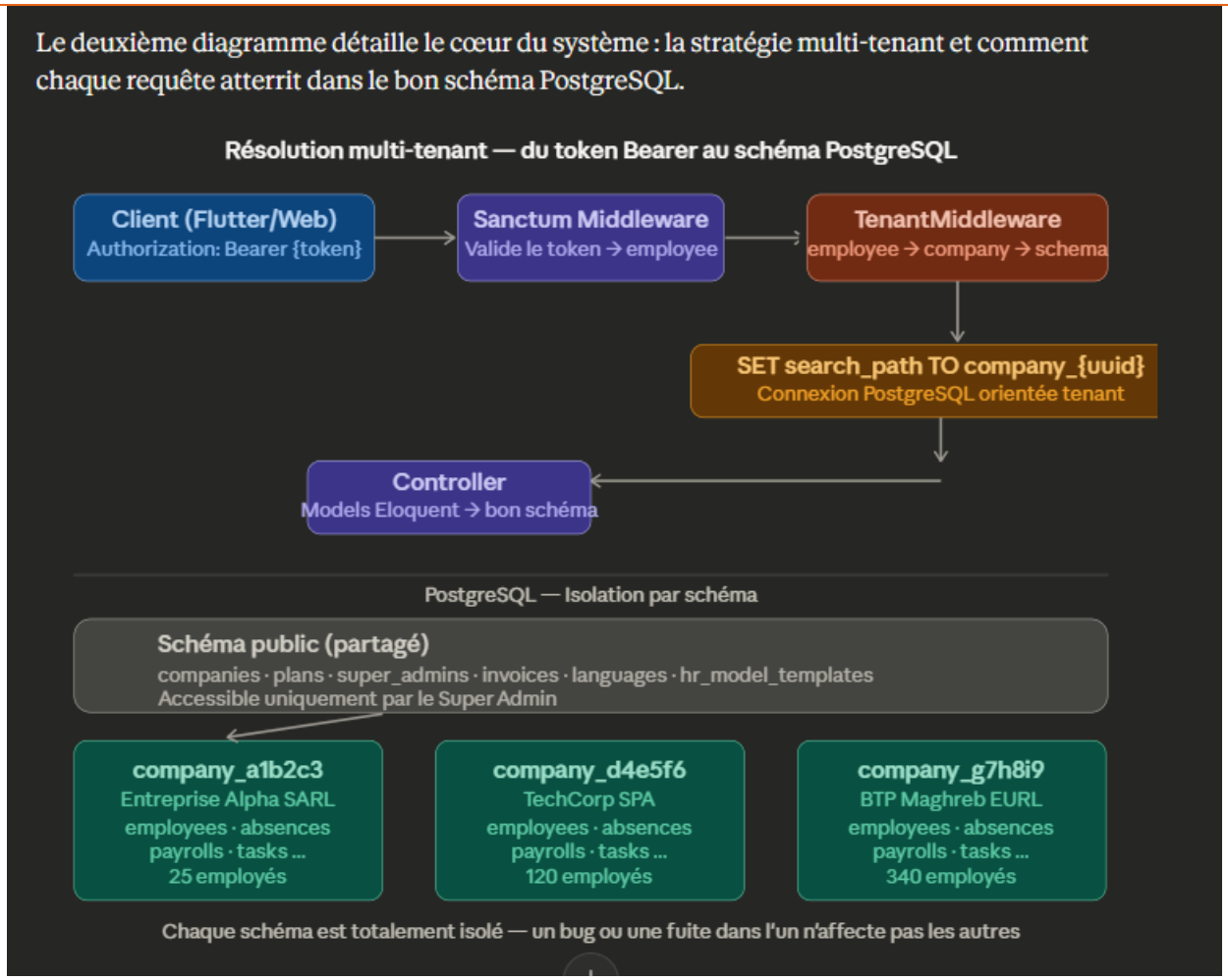
Voici maintenant le flux de pointage déclaratif — le cœur de l'expérience employé au quotidien.

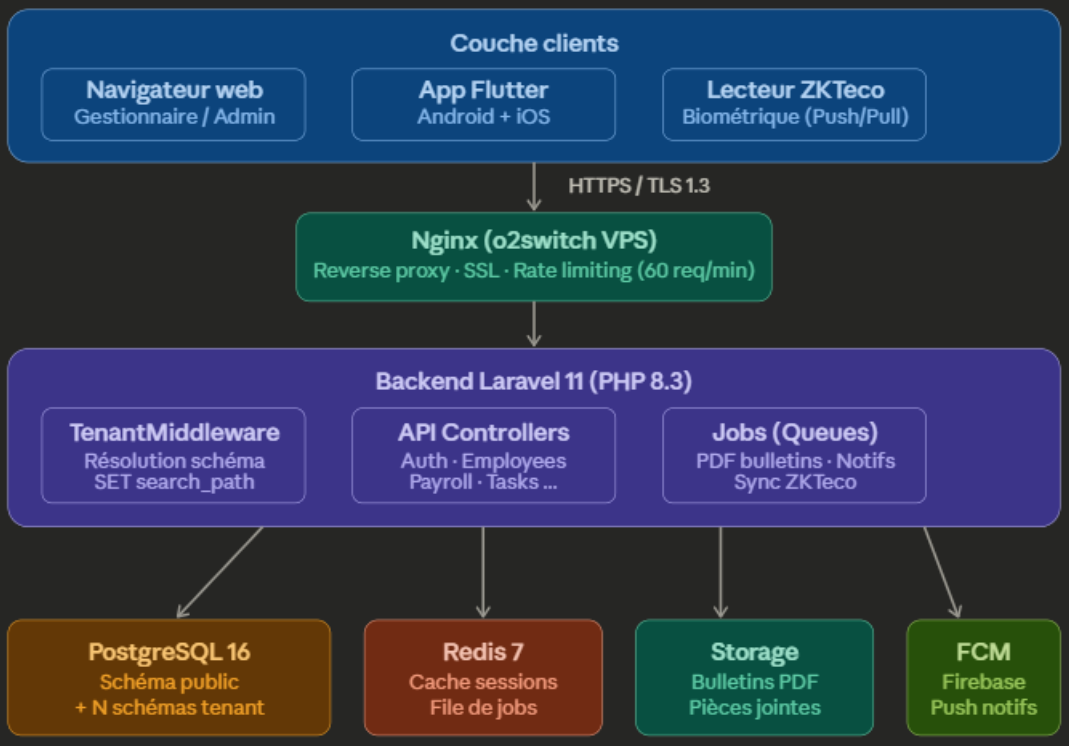


Voici maintenant le flux de pointage déclaratif — le cœur de l'expérience employé au quotidien.



Le deuxième diagramme détaille le cœur du système : la stratégie multi-tenant et comment chaque requête atterrit dans le bon schéma PostgreSQL.



Voici maintenant les diagrammes qui accompagnent le DCT. On commence par l'architecture globale du système.

Le premier diagramme montre comment les couches de Leopardo RH s'organisent — des clients jusqu'aux données.